

Reviewer Pack — Poset Dimension and Karchmer-Wigderson Communication Complex...

Entry 5da693247185 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 10:53:30 UTC

Poset Dimension and Karchmer-Wigderson Communication Complexity for Random CNFs

Entry ID: 5da693247185 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 10:53:30 UTC

1. Conjecture

Field A (mathematical branch): Poset Dimension Theory **Field B** (complexity object): Karchmer-Wigderson Communication Complexity

Statement:

For a random 3-CNF formula with n variables, the communication complexity of the Karchmer-Wigderson game is $\Theta(\text{poset dimension})$, where poset dimension is the minimum number of linear extensions required to represent the clause-variable incidence poset.

Rationale (proposer's reasoning):

The poset dimension captures the structural complexity of the clause-variable relationships, which directly influences the communication required to coordinate the computation in the KW game. This link could reveal hidden hierarchical structures in the formula's truth table.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 09a364f3d297fd7e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import combinations

def generate_random_3cnf(n: int, m: int) -> list:
    variables = set(range(1, n + 1))
    clauses = []
    for _ in range(m):
        clause = [random.choice([var, -var]) for var in random.sample(variables, 3)]
        clauses.append(clause)
    return clauses

def poset_dimension(clauses: list, n: int) -> int:
    poset = {i: set() for i in range(1, n + 1)}
    for clause in clauses:
        for var in clause:
            if -var not in poset[-var]:
                poset[-var].add(var)
    return max(len(poset[var]) for var in poset)

def karchmer_wigderson_communication_complexity(clauses: list, n: int) -> int:
    # Simplified simulation of KW communication complexity
    # This is a placeholder and should be replaced with actual protocol simulation
    return len(clauses)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    m = random.randint(3 * n, 6 * n)
    clauses = generate_random_3cnf(n, m)

    pos_dim = poset_dimension(clauses, n)
    comm_complexity = karchmer_wigderson_communication_complexity(clauses, n)

    return {
        "metric_name": "communication_complexity",
        "metric_value": comm_complexity,
```

```

        "instances_tested": 1,
        "conjecture_holds": pos_dim == comm_complexity,
        "counterexample": "" if pos_dim == comm_complexity else f"Graph with n={n}, A={clauses}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47562927.py", line 61, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47562927.py", line 42, in run_trial
    clauses = generate_random_3cnf(n, m)
               ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_47562927.py", line 21, in generate_random_3cnf
    clause = [random.choice([var, -var]) for var in random.sample(variables, 3)]
               ~~~~~^
  File "/usr/lib/python3.12/random.py", line 413, in sample
    raise TypeError("Population must be a sequence. "
TypeError: Population must be a sequence. For dicts or sets, use sorted(d).

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to `TypeError` in generating random 3-CNF formulas, preventing data collection. | next: Fix the population sequence error in `generate_random_3cnf` by using an ordered list instead of a set/dict

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	137278
2	propose	ollama_remote	qwen3:8b	0	0	108269
3	preregistration	ollama_remote	qwen3:8b	0	0	31639
4	novelty	ollama_remote	qwen3:8b	0	0	28880
5	novelty	ollama_remote	qwen3:8b	0	0	19172
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15320
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9210
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9988
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8065
10	judge	ollama_remote	qwen3:8b	0	0	23203

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 391025 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5da693247185.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5da693247185.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5da693247185.tar.gz` (if generated)